

E-commerce Frontend

The capstone. Build a complete shop that ties together everything: TypeScript, React, routing, a Redux cart, and a validated add-product form.

WHAT WE COVER

Routing

Reusable components

Redux cart

Validated forms

TypeScript

Full build

This is where it all comes together. You'll build a real e-commerce front-end: a product catalog, product detail pages, a shopping cart that works across the whole app, and a form to add new products. Every skill from the roadmap has a job here — components and props, hooks, routing, Redux, forms and TypeScript. This handout is your build plan and reference; work through it section by section.

E-commerce Frontend — Capstone

One project, every skill. We'll wire routing (Day 12), reusable components (Days 10-11), a Redux cart (Day 14) and a validated form (Day 13) into a single typed React app. Follow the sections in order and you'll have a working shop.

CLASS AGENDA • ~2 HOURS

0:00	What we're building & the plan	\$1
0:15	Project structure & the data types	\$2
0:30	Routing & the shared layout	\$3
0:50	Catalog & the reusable ProductCard	\$4
1:10	The Redux cart: add, view, total	\$5-6
1:35	The validated Add-Product form	\$7
1:55	Wire it together & wrap-up	\$8

ON THIS HANDOUT

- | | |
|-------------------------------|-----------------------------|
| 01. What we're building | 05. The cart with Redux |
| 02. Project structure & types | 06. The cart page & total |
| 03. Routing & layout | 07. The Add-Product form |
| 04. The catalog & ProductCard | 08. Putting it all together |

PART 1 • Plan the App

01 What we're building

A small but complete online store with four pages and a working cart. Each feature maps to a skill you already have:

- ▶ **Home & catalog** — a grid of products (components + props, Days 10-11).
- ▶ **Product detail** — one product by URL, e.g. `/products/3` (routing + useParams, Day 12).
- ▶ **Cart** — add/remove items, see the total, shared across pages (Redux, Day 14).
- ▶ **Add product** — a typed, validated form (Day 13) that adds to the catalog.
- ▶ All of it typed end-to-end with **TypeScript** (Days 8-9).

Set it up with the same Vite + React + TypeScript project from Day 10, then add `react-router-dom`, `@reduxjs/toolkit`, `react-redux` and `zod`.

02 Project structure & types

A clean folder layout keeps a bigger app manageable. Group by feature:

```
src/
├─ main.tsx           # Provider + Router mount here
├─ App.tsx            # the route map
├─ types.ts           # shared types (Product, CartItem)
├─ store/
│  ├─ store.ts        # configureStore + persistence
│  ├─ hooks.ts        # typed useAppSelector / useAppDispatch
│  ├─ cartSlice.ts     # cart state + reducers
│  └─ productsSlice.ts # product catalog state
├─ components/
│  ├─ Layout.tsx      # navbar + Outlet + footer
│  └─ ProductCard.tsx # reusable product card
└─ pages/
   ├─ Home.tsx
   ├─ Products.tsx
   ├─ ProductDetail.tsx
   ├─ Cart.tsx
   └─ AddProduct.tsx
```

TERMINAL

Start with the shared **types** — the shape of a product and a cart item drive the whole app:

```
// types.ts
export interface Product {
  id: number;
  name: string;
  price: number;
  image: string;
  category: string;
}

export interface CartItem extends Product {
  quantity: number; // a cart item is a product plus how many
}
```

TS

PART 2 • Build the Pieces

03 Routing & layout

Define the pages and a shared layout with `<Outlet />` (Day 12), so the navbar and cart link appear everywhere.

TSX

```
// App.tsx
import { Routes, Route } from "react-router-dom";
import { Layout } from "../components/Layout";
import { Home, Products, ProductDetail, Cart, AddProduct } from "../pages";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetail />} />
        <Route path="/add" element={<AddProduct />} />
        <Route path="/cart" element={<Cart />} />
        <Route path="*" element={<h1>404 - Not found</h1>} />
      </Route>
    </Routes>
  );
}
```

TSX

```
// components/Layout.tsx
import { Link, Outlet } from "react-router-dom";
import { useAppSelector } from "../store/hooks";

export function Layout() {
  const count = useAppSelector((s) => s.cart.items.length); // live cart count
  return (
    <div>
      <nav>
        <Link to="/">Shop</Link>
        <Link to="/products">Products</Link>
        <Link to="/add">Add product</Link>
        <Link to="/cart">Cart ({count})</Link>
      </nav>
      <main><Outlet /></main>
    </div>
  );
}
```

The navbar's cart count reads straight from the Redux store, so it updates the moment anything is added — from any page.

04 The catalog & ProductCard

A reusable **ProductCard** takes a product as a prop and renders it. The catalog maps over products and reuses the card — the component patterns from Days 10-11.

TSX

```
// components/ProductCard.tsx
import { useAppDispatch } from "../store/hooks";
import { addToCart } from "../store/cartSlice";
import type { Product } from "../types";

export function ProductCard({ product }: { product: Product }) {
  const dispatch = useAppDispatch();
  return (
    <div className="card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>${product.price}</p>
      <button onClick={() => dispatch(addToCart(product))}>Add to cart</button>
    </div>
  );
}
```

TSX

```
// pages/Products.tsx
import { useAppSelector } from "../store/hooks";
import { ProductCard } from "../components/ProductCard";

export function Products() {
  const products = useAppSelector((s) => s.products.items);
  return (
    <div className="grid">
      {products.map((p) => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}
```

05 The cart with Redux

The cart is shared app-wide, so it lives in Redux (Day 14). The slice handles adding (increasing quantity if already present), removing, and clearing.

TS

```
// store/cartSlice.ts
import { createSlice, PayloadAction } from "@reduxjs/toolkit";
import type { Product, CartItem } from "../types";

interface CartState { items: CartItem[]; }
const initialState: CartState = { items: [] };

const cartSlice = createSlice({
  name: "cart",
  initialState,
  reducers: {
    addToCart: (state, action: PayloadAction<Product>) => {
      const existing = state.items.find((i) => i.id === action.payload.id);
      if (existing) {
        existing.quantity += 1; // already in cart -> +1
      } else {
        state.items.push({ ...action.payload, quantity: 1 });
      }
    },
    removeFromCart: (state, action: PayloadAction<number>) => {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
    clearCart: (state) => { state.items = []; },
  },
});

export const { addToCart, removeFromCart, clearCart } = cartSlice.actions;
export default cartSlice.reducer;
```

- ▶ `addToCart` takes a whole `Product` and bumps quantity if it's already there.
- ▶ Because the cart is in the store, the navbar, product pages and cart page all stay in sync.
- ▶ Wire the same localStorage persistence from Day 14 so the cart survives a refresh.

06 The cart page & total

The cart page reads items from the store, computes the total with `useMemo` (Day 11), and lets the user remove items.

TSX

```
// pages/Cart.tsx
import { useMemo } from "react";
import { useAppSelector, useAppDispatch } from "../store/hooks";
import { removeFromCart } from "../store/cartSlice";

export function Cart() {
  const items = useAppSelector((s) => s.cart.items);
  const dispatch = useAppDispatch();

  const total = useMemo(
    () => items.reduce((sum, i) => sum + i.price * i.quantity, 0),
    [items]
  );

  if (items.length === 0) return <p>Your cart is empty.</p>;

  return (
    <div>
      {items.map((i) => (
        <div key={i.id} className="cart-row">
          <span>{i.name} x {i.quantity}</span>
          <span>${i.price * i.quantity}</span>
          <button onClick={() => dispatch(removeFromCart(i.id))}>Remove</button>
        </div>
      ))}
      <strong>Total: ${total}</strong>
    </div>
  );
}
```

07 The Add-Product form

Reuse the Day 13 pattern: a typed, Zod-validated form. On a valid submit, dispatch the new product into the catalog and route back to the products page.

TSX

```
// pages/AddProduct.tsx
import { useState } from "react";
import { z } from "zod";
import { useNavigate } from "react-router-dom";
import { useAppDispatch } from "../store/hooks";
import { addProduct } from "../store/productsSlice";

const schema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters"),
  price: z.number().positive("Price must be greater than 0"),
  category: z.string().min(1, "Choose a category"),
});
type ProductForm = z.infer<typeof schema>;

export function AddProduct() {
  const [form, setForm] = useState<ProductForm>({ name: "", price: 0, category: "" });
  const [errors, setErrors] = useState<Record<string, string[] | undefined>>({});
  const dispatch = useAppDispatch();
  const navigate = useNavigate();

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    const result = schema.safeParse(form);
    if (!result.success) {
      setErrors(z.flattenError(result.error).fieldErrors);
      return;
    }
    dispatch(addProduct({ ...result.data, id: Date.now(), image: "/placeholder.png" }));
    navigate("/products"); // go see the new product
  }

  return (
    <form onSubmit={handleSubmit}>
      <input placeholder="Name" value={form.name}
        onChange={(e) => setForm({ ...form, name: e.target.value })} />
      {errors.name && <p className="error-text">{errors.name[0]}</p>}

      <input type="number" placeholder="Price" value={form.price}
        onChange={(e) => setForm({ ...form, price: Number(e.target.value) })} />
      {errors.price && <p className="error-text">{errors.price[0]}</p>}

      <button type="submit">Add product</button>
    </form>
  );
}
```

- ▶ The Zod schema validates and types the form in one place (Day 13).
- ▶ On success we `dispatch(addProduct(...))` into the store and `navigate` to the catalog.
- ▶ The new product appears instantly because the products list reads from Redux.

PART 3 • Tie It Together

08 Putting it all together

Finally, mount everything: the Redux **Provider** and the **Router** wrap the app, seeding the store with some initial products.

```
// main.tsx
import { Provider } from "react-redux";
import { BrowserRouter } from "react-router-dom";
import { store } from "../store/store";
import App from "../App";

export default function Root() {
  return (
    <Provider store={store}>           {/* Redux available everywhere */}
      <BrowserRouter>                {/* routing available everywhere */}
        <App />
      </BrowserRouter>
    </Provider>
  );
}
```

How the data flows

- ▶ A user opens `/products` → `Products` reads the catalog from Redux and renders `ProductCard`s.
- ▶ They click “Add to cart” → `dispatch(addToCart)` updates the store → the navbar count and cart page update everywhere.
- ▶ They open `/add`, submit a valid product → it's validated by Zod, dispatched, and they're routed to see it.
- ▶ Refresh the page → the cart is still there, thanks to localStorage persistence.

That's a complete front-end application — the same architecture real shops use, built entirely from skills you now have.

Stretch Goals

Take the capstone further:

- ▶ Add quantity + / - buttons on each cart row (dispatch an `updateQuantity` action).
- ▶ Filter the catalog by category using `useSearchParams` (`/products?category=shoes`).
- ▶ Add a search box on the products page (reuse the Day 11 pattern).
- ▶ Show a “Checkout” page that clears the cart and shows an order summary.
- ▶ Load the initial products from a JSON Server (Day 7) with `useEffect` + `fetch`.
- ▶ Deploy it (Vite `npm run build`) and share the link.

Congratulations — you've built a full front-end

You just assembled a complete, typed e-commerce front-end from the ground up: routing with shared layouts, reusable components, a global Redux cart with persistence, and a validated form — all in TypeScript. This is the exact toolkit professional React developers use every day.

Look back at how far you've come: from HTML and CSS, through JavaScript, the DOM, async, Node, TypeScript, and React, to a real application. Keep building — pick one stretch goal, then start a project of your own. That's how it sticks.

QUICK RECAP

The capstone unites everything: **TypeScript** types · **routing** with a shared layout · reusable **components & props** · a global **Redux cart** with persistence · a **Zod-validated** add-product form. Flow: browse → add to cart → the whole app stays in sync.